Slalom Challenge: Backward DP Planning with Model-Based Bounds and Path Tracking

MD SYMUM HOSSAIN (ID: 20264789)
(MO89905) Learning-Based Control for Mobility Systems
Cho Chun Shik Graduate School of Mobility, KAIST
Fall 2026

TABLE I
OFFICIAL RULES, GIVEN SUGGESTED VALUES AND MY CHOICES

Constraints	Source	Value used
Cone clearance; speed limit	rule	> 0; 12 m/s
Easy gate margin	mine	2.0 m (rule: > 0)
Easy turn bounds (suggested)	mine	$0.31 \to 0.5$ rad; $6 \to 8$ m/s^2
Full speed: planner / controller cap	mine	11.8 / 11.6 m/s
Full body / road margin	mine	0.4 / 0.8 m
Path smoothing; lookahead L_d	mine	5 m; $0.5 + 0.1 v_x$ m
Selection buffer	mine	clearance ≥ 0.3 m

I. PROBLEM STATEMENT

Task– A simulated BMW E90 (PyChrono 10.0.0, $\mu = 0.9$) must pass eight cones at $X_j = 20 + 15j$ m, $Y_j = 0$ (cone 0 on the $+Y$ side, cone 1 on the $-Y$ side, and so on) and reach $X_{\mathrm{REF}} = 145$ m in the shortest time. I solved both modes with the same idea. **The result to evaluate is Full mode (14.696 s)**; Easy mode (16.333 s) is submitted as supporting work.

Official problem– Minimize the finish time t_f. A run fails if the 4.7×1.9 m body touches a cone or leaves the road ($|Y| \leq 5.5$ m), passes a cone on the wrong side ($\sigma_j(Y_{\mathrm{REF}}(t_j) - Y_j) > 0$ is required), breaks $v_x, \dot{X}_{\mathrm{REF}} \leq 12$ m/s, $v_x \geq -1$ m/s or $|\psi| \leq 1.3$ rad, or takes over 60 s. No safety distance is prescribed.

State and action– In Full mode the controller reads $s = (X_{\mathrm{REF}}, Y_{\mathrm{REF}}, \psi, v_x, v_y, r, \delta)$ every 0.02 s and returns steering $u_s \in [-0.8, 0.8]$ and acceleration $a_x \in [-7, 7]$ m/s^2. A fixed input adapter limits the steering change to 0.04 per 0.02 s step and converts a_x to throttle or brake. In Easy mode the action is a list of moves $m \in \{1..5\}$, $n \in \{-4..4\}$ on a 1×0.5 m grid (0.5 s stages), driven by a fixed tracker.

My objective and its relation to the official one– I plan the REF point with dynamic programming (DP). Each 0.5 s stage costs 1 (in Full mode the last stage costs only the fraction needed to reach the line), so the DP minimizes the planned time, and every rule becomes a forbidden move. A controller must still follow the plan, so I judged every design by its PyChrono finish time and measured clearance (Table I).

II. PROPOSED APPROACH

Backward DP (both modes)– An offline DP plans a minimum-time path; an online controller follows it (Fig. 1). The DP state is $z = (X, Y, p)$ with p the previous move, because the speed and turning rules compare each move with

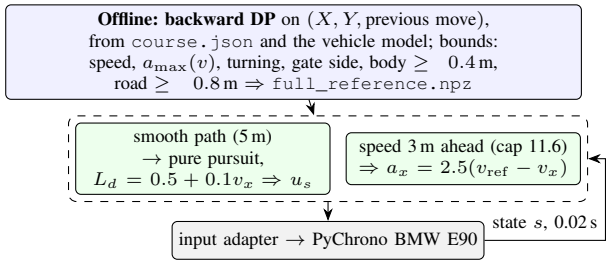


Fig. 1. My Full-mode method: an offline backward-DP planner and an online path-tracking controller. Easy mode uses the same DP on the given grid, with the given tracker.

the one before (state augmentation). Every move increases X, so the Bellman equation

$$J(z) = \min_{u \in U(z)} \big[g(z, u) + J(f(z, u)) \big], \tag{1}$$

with $J = 0$ for $X \geq 145$, is solved backward from the finish; $U(z)$ holds only allowed moves (a cone crossing is interpolated and its side offset checked). The best move per state is a policy table, followed forward from the start. With the suggested bounds my DP gives exactly the provided results (48 stages for a 2.5 m gate margin, 31 for 0.5 m).

Easy mode– The grid is fixed, so I tuned only the planning bounds. Two suggested bounds limit turning per stage, $|\Delta\chi| \leq c$ and $|\Delta\chi| \leq a_{\mathrm{lat}} \Delta / V$, and the smaller one is active. At 10 m/s, $(c, a_{\mathrm{lat}}) = (0.31, 6)$ allows only 0.30 rad per stage and $(0.5, 8)$ allows 0.40 rad; with a 2.0 m gate margin this gives 32 stages (16.0 s planned).

Full-mode planner– I use my own finer grid: 0.25×0.25 m cells, $m \leq 23$ and $|n| \leq 8$ (391 moves, speed steps of 0.5 m/s). New bounds: (i) the time to accelerate from V_{k-1} to V_k, $\int dv / a_{\max}(v)$ with the input adapter's achievable $a_{\max}$, must fit in one stage; (ii) the car rectangle, placed along each move, must stay ≥ 0.4 m from every cone (scoring-code formula); (iii) and ≥ 0.8 m inside the road edge. With $V \leq 11.8$ m/s and $a_{\mathrm{lat}} \leq 8$ m/s^2, the DP finds 29 stages (14.33 s planned). Path, speeds and the fitted steering map are saved in `full_reference.npz`, the only file the controller loads.

Full-mode controller– The path corners are rounded with a 5 m moving average; then pure pursuit steers the rear axle towards a path point $L_d = 0.5 + 0.1 v_x$ ahead, with body-frame

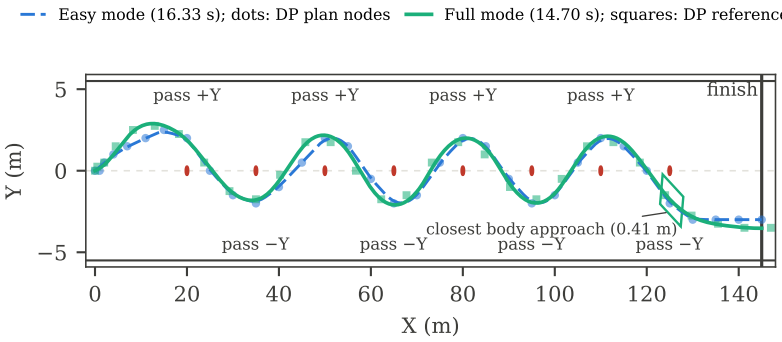


Fig. 2. Measured REF paths of my final solutions (replicated runs; lateral scale about 3×; dots/squares: Easy plan and Full reference nodes). The rectangle marks the closest body approach of the Full run (0.41 m), just after REF has passed cone 7.

TABLE II
PyChrono results (all eight gates passed)

Solution	time (s)	min clearance (m)
Example controller (Full, given)	30.660	0.881
My DP, suggested bounds, 48 stages (Easy)	24.046	0.738
My final Easy plan, 32 stages	**16.333**	**0.440**
My final Full controller	**14.696**	**0.414**

offset (l_x, l_y):

$$\delta = \arctan \frac{2L\, l_y}{l_x^2 + l_y^2}, \qquad u_s = \frac{\delta}{s_g + s_v v_x^2}, \tag{2}$$

with $L = 2.776\,\text{m}$ and the given fitted steering map $s_g = 0.5514$, $s_v = -4.2 \cdot 10^{-4}$. Speed is tracked with $a_x = 2.5\,(v_{\text{ref}} - v_x)$, where v_{ref} is the planned speed 3 m ahead, capped at 11.6 m/s (rule: 12). One call takes about 0.05 ms.

III. Own Validation

Results– Table II gives the final results on my PC. Tuning sweeps and the Full-mode extra checks used the same files on a second (Linux) installation; the machines agreed within 10 ms and 0.9 mm. The Full controller is about 2.1 times faster than the example controller and 1.64 s faster than my Easy plan. It passes a cone every 1.37 s (≈11.0 m/s) and finishes at 11.54 m/s (Fig. 2); a cone-free straight run with the same 11.6 m/s cap would need about 14.0 s.

How the final values were chosen– I took the fastest PyChrono run with clearance ≥ 0.3 m whose neighbouring settings also kept ≥ 0.3 m. I fixed this buffer in advance, because a crash gives the minimum score while small setting changes moved the clearance by 0.1–0.2 m. All ten neighbours of the final Full controller (smoothing 4/6 m, lookahead gain 0.05/0.15, body margin 0.35/0.45 m, speed gain 2.0/3.0, preview 2/4 m) succeeded with 14.666–14.785 s and 0.333–0.477 m.

Failures and lessons– (1) Easy: relaxing the turn bounds made the plan faster *and* safer (19.655 s, 0.328 m → 16.333 s, 0.440 m), but every 31-stage plan hit a cone because the fixed tracker cuts corners (planned 2.0 m, measured 1.85–1.90 m). (2) Full: a 5.5 m lookahead cut a corner and hit cone 2. (3) At ≈11 m/s a short lookahead made the steering oscillate up to ±0.8 until the car left the road; the sharp DP path corners were the main cause (with 5 m smoothing all tested gains 0.05–0.15 were stable). (4) With the smoothed tracker, $a_{\text{lat}} = 9\,\text{m/s}^2$ led to road departure (3 of 3 runs), and on the final setup a doubled speed gain (5.0) led to cone contact. (5) Learning a lateral path correction from repeated runs made the car follow the DP path closely, but each corrected run was below the buffer (0.16–0.29 m) or slower: the body check assumes the car points along each segment, while the real heading differs from the travel direction of REF by up to 8.9°.

Extra checks– When all cones were moved towards the path, the Full controller finished at a 0.40 m shift and touched at 0.45 m (Easy, my PC: 0.4/0.5 m). Peaks stayed inside the rules ($v_x = 11.62$ m/s, $\dot{X}_{\text{REF}} = 11.57$ m/s, $|\psi| = 0.42$ rad, road clearance ≥ 1.0 m). The supplied model's lateral error on the final run was 0.035/0.119/0.381 m at 0.5/1/2 s: good only for short horizons.

Limitations– The controller ignores the cones online (a new course needs re-planning), the body check and lateral-acceleration bound are approximations (peak ≈9.2 m/s^2 for a bound of 8), and the supplied model (≈0.3 ms/call) was too slow for an online rollout.

IV. Conclusion

One backward-DP method with model-based bounds and exact body and road checks solved both modes: 16.333 s (Easy) and **14.696 s (Full)** with 0.414 m clearance, against 30.660 s for the example controller. A plan that is feasible on paper must be tested on the plant, and the tracker decides how aggressive the plan can be. Next step: a rollout with this controller as base policy, using a fast model and the real cone positions.

AI use: I used ChatGPT (GPT-5.6 Luna, default) to interpret and formulate the problem, and Claude (Anthropic, Claude app; configured model claude-opus-5-5) as a coding assistant: code drafting and debugging, replication runs on a separate Linux installation, tuning, visualizing etc. I directed the work (mode order, robustness, optimizing, speed–safety trade-off), ran all code on my own PC and reviewed all AI-generated content; the final results were reproduced on my PC, and I take full responsibility for this report.

Find all the relevant code & implementations at: https://github.com/SymumAust/slalom-challenge-2026-symum